

CH 1.1: SUPERVISED LEARNING: BASIC METHODS & LINEAR MODELS

Contents

Supervised Learning	1
1. Classification (Discrete Output)	1
2. Regression (Continuous Output)	2
Types of Distances	3
K-Nearest Neighbours (KNN) method	5
Decision Trees	6
ID3 Algorithm (Iterative Dichotomiser 3)	7
Entropy in Decision Trees	7
Information Gain in Decision Trees	7
CART Algorithm (Classification and Regression Trees)	11
Impurity Measures.....	11
Regression Tree (type of decision tree).....	16
Naïve Bayes	18
Bayes Classifier.....	18
Naïve Bayes Classifier.....	18

Supervised Learning

Supervised Learning is a fundamental branch of machine learning where an algorithm learns a function that maps an input to an output based on **labeled examples**. The core idea is to train the model with a dataset where the "answer" or **target variable** is already known.

Once trained, the model can predict the output for new, unseen data. Supervised learning is broadly divided into two main categories based on the nature of the target variable: **Regression** and **Classification**.

1. Classification (Discrete Output)

Classification is a supervised learning task where the goal is to predict a **discrete class label** or category for a given input. **Key Characteristics:**

- **Target Variable:** Categorical (e.g., A or B, High/Medium/Low, True/False, Dog/Cat/Bird).
- **Objective:** To find a decision boundary that best separates data points belonging to different classes in the feature space.

- **Evaluation Metrics:** Accuracy, Precision, Recall, F1-Score, and the AUC-ROC Curve.

Examples of Classification Problems:

- **Binary Classification:** Predicting one of two classes (e.g., spam detection: *Spam* or *Not Spam*; medical diagnosis: *Disease* or *No Disease*).
- **Multi-class Classification:** Predicting one of more than two classes (e.g., image recognition: *Cat*, *Dog*, or *Bird*; letter recognition: A, B, C, etc.).

Common Classification Algorithms:

- **Logistic Regression:** A linear model used to estimate the probability of a binary outcome (despite the name, it's a classifier).
- **k-Nearest Neighbors (k-NN):** Assigns a class based on the majority vote of the k closest data points.
- **Decision Trees:** Partitions the data space into a set of non-overlapping regions and assigns a class label to each region.
- **Support Vector Machines (SVM):** Finds a hyperplane that maximizes the margin between the different classes.
- **Naive Bayes:** A probabilistic classifier based on applying Bayes' theorem with strong (naive) independence assumptions between the features.

2. Regression (Continuous Output)

Regression is a supervised learning task where the goal is to predict a **continuous numerical value** for a given input. **Key Characteristics:**

- **Target Variable:** Continuous, real-valued number (e.g., 1.5, 100.25, -5.0).
- **Objective:** To find a function (often a line or curve) that best fits the data, minimizing the distance between the function and the actual data points.
- **Evaluation Metrics:** **Mean Squared Error (MSE)**, Root Mean Squared Error (RMSE), and Mean Absolute Error (MAE).

Examples of Regression Problems:

- **Price Prediction:** Predicting the selling price of a house based on its features (size, location, age).
- **Forecasting:** Predicting future stock prices, temperature, or sales volume.
- **Estimation:** Estimating a person's age or weight based on health metrics.

Common Regression Algorithms:

- **Linear Regression:** Fits a straight line (or hyperplane in multiple dimensions) to the data to model the linear relationship between input features and the target variable.
- **Polynomial Regression:** Models a curvilinear relationship by using polynomial features of the input variable(s).
- **k-Nearest Neighbors (k-NN) Regression:** Predicts the value based on the **average** of the k closest data points' values.

- **Support Vector Regression (SVR):** A variation of SVM that finds a function that has at most ϵ (epsilon) deviation from the actual target values.

Feature	Classification	Regression
Goal	To predict a discrete class label or category.	To predict a continuous numerical value .
Output Type	Categorical/Discrete (e.g., <i>Class A</i> , <i>Class B</i> , 0 or 1, True/False).	Continuous/Real-Valued (e.g., any number like \$10.5, 99.9, -5.2\$).
Nature of Problem	Involves finding an optimal decision boundary to separate distinct groups.	Involves finding an optimal function/curve to fit the data points.
Examples	Spam detection, Image recognition, Disease diagnosis.	House price prediction, Stock price forecasting, Temperature prediction.
Primary Error Metric	Accuracy , Precision, Recall, F1-Score, AUC-ROC.	Mean Squared Error (MSE) , Root Mean Squared Error (RMSE), Mean Absolute Error (MAE).
Common Algorithms	Logistic Regression, Decision Trees, K-Nearest Neighbors (k-NN), Support Vector Machines (SVM).	Linear Regression, Polynomial Regression, Support Vector Regression (SVR), K-Nearest Neighbors (k-NN).

Types of Distances

1. **Euclidean Distance:** The most common distance metric, used to compute the straight-line distance between two points in Euclidean space.

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Given two points $\mathbf{x} = (3, 4)$ and $\mathbf{y} = (0, 0)$:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{(3-0)^2 + (4-0)^2} = \sqrt{9+16} = \sqrt{25} = 5$$

2. **Manhattan Distance (L1 Norm):** Computes the sum of the absolute differences between corresponding coordinates. Also known as **taxicab** or **city block distance**.

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n |x_i - y_i|$$

For points $\mathbf{x} = (3, 4)$ and $\mathbf{y} = (0, 0)$:

$$d(\mathbf{x}, \mathbf{y}) = |3-0| + |4-0| = 3 + 4 = 7$$

3. **Minkowski Distance:** A generalization of Euclidean and Manhattan distances. It's defined by a parameter p which controls the type of distance:

- When $p=1$, it becomes **Manhattan Distance**.
- When $p=2$, it becomes **Euclidean Distance**.

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}}$$

For $p = 3$, $\mathbf{x} = (3, 4)$, and $\mathbf{y} = (0, 0)$:

$$d(\mathbf{x}, \mathbf{y}) = (|3 - 0|^3 + |4 - 0|^3)^{\frac{1}{3}} = (27 + 64)^{\frac{1}{3}} = (91)^{\frac{1}{3}} \approx 4.49$$

4. **Cosine Similarity:** Measures the cosine of the angle between two vectors. It's often used for text data, where vectors represent word frequencies or term weights.

$$\text{cosine similarity} = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

For vectors $\mathbf{x} = (1, 2, 3)$ and $\mathbf{y} = (4, 5, 6)$:

$$\mathbf{x} \cdot \mathbf{y} = 1 \times 4 + 2 \times 5 + 3 \times 6 = 32$$

$$\|\mathbf{x}\| = \sqrt{1^2 + 2^2 + 3^2} = \sqrt{14}, \quad \|\mathbf{y}\| = \sqrt{4^2 + 5^2 + 6^2} = \sqrt{77}$$

$$\text{cosine similarity} = \frac{32}{\sqrt{14} \times \sqrt{77}} \approx 0.974$$

5. **Hamming Distance:** Used primarily for categorical variables, it counts the number of positions at which the corresponding elements of two strings are different.

For strings $\mathbf{x} = "1101"$ and $\mathbf{y} = "1011"$:

$$\text{Hamming distance} = 2 \quad (\text{positions 2 and 3 are different})$$

6. **Chebyshev Distance:** Similar to Manhattan distance but takes the maximum difference along any coordinate dimension.

$$d(\mathbf{x}, \mathbf{y}) = \max(|x_i - y_i|)$$

For $\mathbf{x} = (3, 4)$ and $\mathbf{y} = (0, 5)$:

$$d(\mathbf{x}, \mathbf{y}) = \max(|3 - 0|, |4 - 5|) = \max(3, 1) = 3$$

7. **Mahalanobis Distance:** A distance metric that accounts for the correlations of the data set and is used for normalized data.

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y})^T S^{-1} (\mathbf{x} - \mathbf{y})}$$

Consider $\mathbf{x} = (2, 3)$, $\mathbf{y} = (1, 1)$, and the covariance matrix $S = \begin{bmatrix} 2 & 0.5 \\ 0.5 & 1 \end{bmatrix}$. First, compute the inverse of S :

$$S^{-1} = \begin{bmatrix} 1.2 & -0.6 \\ -0.6 & 2 \end{bmatrix}$$

Now calculate the Mahalanobis distance:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y})^T S^{-1} (\mathbf{x} - \mathbf{y})}$$

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\begin{bmatrix} 1 \\ 2 \end{bmatrix}^T \begin{bmatrix} 1.2 & -0.6 \\ -0.6 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix}}$$

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{2.4 + 4.8} = \sqrt{7.2} \approx 2.68$$

K-Nearest Neighbours (KNN) method

KNN is a **supervised, non-parametric, lazy learning algorithm** used for both **classification** and **regression**.

1. **Choice of k:** k is a parameter that refers to the number of nearest neighbours considered when predicting the class of a new data point. A small k may lead to overfitting; a large k can smooth the prediction but may miss local patterns.
2. **Type:** Supervised learning algorithm — used for classification and regression.
3. **Nature:** Non-parametric and lazy — no model is built during training.
4. **Training:** No explicit training phase — stores the dataset.
5. **Prediction:** Based on majority vote (classification) or average (regression) of k nearest neighbors.
6. **Distance metric:** Typically uses Euclidean distance, but others like Manhattan or Minkowski can also be used.
7. **Feature scaling:** Necessary — unscaled features can distort distance calculations.

Example: Predict BMI category for height= 165 and weight =75 when value of k =

- i) 3
- ii) 5
- iii) 7

Height (cm)	Weight (kg)	BMI Category
160	55	Normal weight
165	65	Normal weight
170	70	Normal weight
175	85	Overweight
180	90	Overweight
185	110	Obesity
190	115	Obesity
150	40	Underweight
155	50	Underweight
200	120	Obesity

Answer

Height	Weight	BMI Category	Distance Calculation	Rank
160	55	Normal weight	$\sqrt{((165-160)^2 + (75-55)^2)} = \sqrt{(25 + 400)} \approx \mathbf{20.62}$	4
165	65	Normal weight	$\sqrt{((165-165)^2 + (75-65)^2)} = \sqrt{(0 + 100)} = \mathbf{10.00}$	2
170	70	Normal weight	$\sqrt{((165-170)^2 + (75-70)^2)} = \sqrt{(25 + 25)} \approx \mathbf{7.07}$	1
175	85	Overweight	$\sqrt{((165-175)^2 + (75-85)^2)} = \sqrt{(100 + 100)} \approx \mathbf{14.14}$	3
180	90	Overweight	$\sqrt{((165-180)^2 + (75-90)^2)} = \sqrt{(225 + 225)} \approx \mathbf{21.21}$	5
185	110	Obesity	$\sqrt{((165-185)^2 + (75-110)^2)} = \sqrt{(400 + 1225)} \approx \mathbf{40.31}$	8
190	115	Obesity	$\sqrt{((165-190)^2 + (75-115)^2)} = \sqrt{(625 + 1600)} \approx \mathbf{47.17}$	9
150	40	Underweight	$\sqrt{((165-150)^2 + (75-40)^2)} = \sqrt{(225 + 1225)} \approx \mathbf{38.08}$	7
155	50	Underweight	$\sqrt{((165-155)^2 + (75-50)^2)} = \sqrt{(100 + 625)} \approx \mathbf{26.93}$	6
200	120	Obesity	$\sqrt{((165-200)^2 + (75-120)^2)} = \sqrt{(1225 + 2025)} \approx \mathbf{57.01}$	10
165	75			

k	Top k Persons	Predicted BMI Category
3	N, N, OW	Normal weight
5	N, N, OW, N, OW	Normal weight
7	N, N, OW, N, OW, UW, UW	Normal weight

Decision Trees

A Decision Tree is a flowchart-like tree structure where each internal node represents a test on an attribute, each branch represents the outcome of the test, and each leaf node represents a class label (decision).

- **Root Node:** The top node that represents the entire dataset; splits based on the most significant attribute.
- **Internal Nodes:** Represent decisions based on attribute tests.
- **Branches:** Represent the outcome of the test at a node (e.g., yes/no or categorical values).
- **Leaf Nodes (Terminal Nodes):** Represent the final class label or output.
- **Popular algorithms:**
 - ID3 (Iterative Dichotomiser 3)
 - C4.5 / C5.0 (improvement over ID3)
 - CART (Classification and Regression Trees)
- **Pruning (Tree Simplification):** Pruning helps avoid **overfitting** by removing unnecessary branches:
 - **Pre-Pruning:** Stop tree growth early (e.g., minimum samples per node).
 - **Post-Pruning:** Build full tree first, then remove parts using validation data.
- **Handling Different Types of Data**
 - **Categorical Attributes:** Directly split based on category values.
 - **Numerical Attributes:** Use thresholds (e.g., Age $\leq$ 30, Age $>$ 30) to create binary splits.

Splitting Criteria: Used to determine the "**best**" attribute to split at each node

a. Information Gain (ID3)

- Measures **reduction in entropy** after the split.
- Higher information gain $\rightarrow$ better attribute.

b. Gain Ratio (C4.5)

- Adjusts Information Gain by considering **intrinsic information** (to avoid bias toward attributes with many values).

c. Gini Index (CART)

- Measures **impurity** of a node; lower Gini $\rightarrow$ better split.

Algorithm	Splitting Criteria	Handles Missing Values	Pruning	Output Type
ID3	Information Gain	No	No	Classification
C4.5/C5.0	Gain Ratio	Yes	Yes	Classification
CART	Gini Index	Yes	Yes	Classification & Regression

Advantages of Decision Trees

- Easy to understand and interpret (visual flow of decisions).
- Handles both categorical and numerical data.
- Requires little data preprocessing (no need for normalization).
- Performs well with non-linear data.
- Non-metric: Doesn't rely on distance functions.

Limitations

- Prone to overfitting, especially on noisy data.
- Can be unstable: small changes in data can result in different trees.
- Biased towards attributes with more levels (values) if using plain information gain.

Example:

A decision tree might start with:

IF Weather = Sunny THEN Play = No

ELSE IF Weather = Overcast THEN Play = Yes

ELSE IF Weather = Rainy THEN ...

Weather	Temperature	Play
Sunny	Hot	No
Overcast	Mild	Yes
Rainy	Cool	Yes

ID3 Algorithm (Iterative Dichotomiser 3)

- Used for generating a decision tree from a set of labeled training data
- Works only with **categorical attributes** (numerical attributes must be discretized).
- Handles only **classification** (not regression).
- Assumes **complete data** (no missing values).

Entropy in Decision Trees

Entropy is a measure of **impurity** or **uncertainty** in a dataset. In the context of decision trees, it tells us how **mixed** the classes are at a particular node.

- If all examples belong to one class, entropy = 0 (pure).
- If examples are evenly distributed among all classes, entropy is maximum.

For a dataset S with classes $C_1, C_2, \dots, C_k$:

- p_i = proportion of samples in class C_i
- $\log_2$ is used to measure in **bits**

$$\text{Entropy}(S) = - \sum_{i=1}^k p_i \log_2 p_i$$

Information Gain in Decision Trees

Information Gain (IG) measures the reduction in entropy after a dataset is split on an attribute. It tells us how much “information” a feature gives us about the class.

- High IG = the attribute **reduces uncertainty** a lot $\rightarrow$ good for splitting.
- Low IG = the attribute **adds little value** $\rightarrow$ not preferred.
- S = current dataset (parent node)
- A = attribute on which the split is made
- v = a possible value of attribute A
- S_v = subset of S where attribute $A = v$
- $|S|$ = total number of samples
- $|S_v|$ = number of samples in subset S_v

$$\text{Information Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \cdot \text{Entropy}(S_v)$$

Advantages of ID3

- **Simple and intuitive.**

- Builds **interpretable trees**.
- Works well with **categorical data**.

7. Disadvantages

- Can **overfit** if the tree is too deep.
- **Biased** toward attributes with many values.
- Doesn't handle **numeric data** directly.
- No **pruning** mechanism (ID3 grows full tree).

Example:

Age	Competition	Type	Profit (class)
Old	Yes	SW	Down
Old	No	SW	Down
Old	No	HW	Down
Mid	Yes	SW	Down
Mid	Yes	HW	Down
Mid	No	HW	Up
Mid	No	SW	Up
New	Yes	SW	Up
New	No	HW	Up
New	No	SW	Up

a) Draw Decision tree using ID3 algorithm

b) Find the class label for the data: [Age: Mid, Competition: Yes, Type: HW]

Answer

① Attribute Age

Values(Age) = {Old, Mid, New}

$S(\text{Age}) = [5 \text{ Down}, 5 \text{ Up}]$

$\text{Entropy}(S) = -\frac{5}{10} \log(\frac{1}{2}) - \frac{5}{10} \log(\frac{1}{2}) = 1$

② $S_{\text{Old}} = [3 \text{ D}, 0 \text{ U}]$

$E(S_{\text{Old}}) = -\frac{3}{3} \log(1) - 0 = 0$

③ $S_{\text{Mid}} = [2 \text{ D}, 2 \text{ U}]$ $E = -\frac{2}{4} \log(\frac{1}{2}) - \frac{2}{4} \log(\frac{1}{2}) = 1$

④ $S_{\text{New}} = [0 \text{ D}, 3 \text{ U}]$ $E = 0$

$\text{Gain}(S, \text{Age}) = 1 - (\frac{3}{10} \times 0 + \frac{4}{10} \times 1 + \frac{0}{10} \times 0) = 0.6$

② Attribute Competition

Value(Com) = {Yes, No}

$$\textcircled{i} S_{\text{yes}} = [3A, 1Y], \quad E = -\frac{3}{4} \log\left(\frac{3}{4}\right) - \frac{1}{4} \log\left(\frac{1}{4}\right) \\ = +0.31127 + 0.5 = 0.81127$$

$$\textcircled{ii} S_{\text{no}} = [2D, 4Y], \quad E = -\frac{2}{6} \log\left(\frac{1}{3}\right) - \frac{4}{6} \log\left(\frac{2}{3}\right) \\ = 0.91829$$

$$\text{Gain}(S, \text{Comp}) = 1 - \left(\frac{4}{10} \times 0.81127 + \frac{6}{10} \times 0.91829 \right) \\ = 1 - (0.32451 + 0.55097) \\ = 0.12452$$

③ Attribute Type

V(Type) = {SW, HW}

$$S_{\text{SW}} = [3D, 3U], \quad E = -\frac{3}{6} \log\left(\frac{1}{2}\right) - \frac{1}{2} \log\left(\frac{1}{2}\right) = 1$$

$$S_{\text{HW}} = [2D, 2U], \quad E = -\frac{2}{4} \log\left(\frac{1}{2}\right) - \frac{1}{2} \log\left(\frac{1}{2}\right) = 1$$

$$\text{Gain}(S, \text{Type}) = 1 - (0.6 \times 1 + 0.4 \times 1) = 0$$

As Gain(S, Age) is highest, Age is root node



Age	Comp	Type	Profit
Mid	Yes	SW	Down
Mid	Yes	HW	Down
Mid	No	HW	Up
Mid	No	SW	Up

$$\text{Value}(\text{mid}) = [\text{Down}, \text{Up}]$$

$$E(\text{mid}) = \frac{2}{4} \log\left(\frac{1}{2}\right) - \frac{1}{2} \log\left(\frac{1}{2}\right) = 1$$

① Comp

$$\text{Value}(C) = \text{Yes, No}$$

$$\textcircled{1} S_y = [2D, 0U], E = \frac{2}{2} \log(1) - 0 = 0$$

$$\textcircled{2} S_n = [2U, 0D], E = 0$$

$$\text{gain}(S_{\text{mid}}, \text{Comp}) = 1 - 0 = 1$$

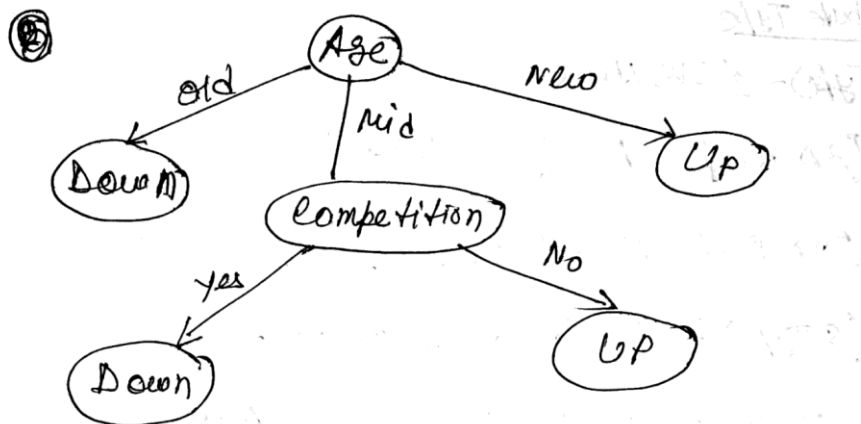
② Type

$$\text{Value}(T) = \{SW, HW\}$$

$$\textcircled{1} S_{SW} = [1D, 1U], E = \frac{1}{2} \log\left(\frac{1}{2}\right) - \frac{1}{2} \log\left(\frac{1}{2}\right) = 1$$

$$\textcircled{2} S_{HW} = [1D, 1U], E = 1$$

$$\text{gain}(S_{\text{mid}}, \text{Type}) = 1 - (0.5 \times 1 + 0.5 \times 1) = 0$$



④ Also we need to predict class label for the data
 $[\text{Age} = \text{mid}, \text{Competition} = \text{Yes}, \text{Type} = \text{HW}]$

Ans. Class label is Down

CART Algorithm (Classification and Regression Trees)

- Used for constructing **binary decision trees** for **both classification and regression** tasks.
- **Binary Trees Only**: Each internal node splits into exactly two branches (unlike ID3/C4.5 which can have multiple).
- Can handle both:
 - **Classification problems** (discrete output)
 - **Regression problems** (continuous output)
- Supports both **numerical** and **categorical** attributes.
- Uses **Gini Index** (for classification) instead of Entropy.
- **Split the dataset** recursively using a feature and a threshold that results in the **most “pure” subsets**.
- Use different **impurity measures** for classification and regression.

Handling Data Types

- **Numerical Features**: CART finds the best **threshold** (e.g., Age < 35).
- **Categorical Features**: Handles binary split (e.g., Type = HW or not).
- **Missing Values**: Can handle using surrogate splits.

Impurity Measures

A. For Classification: Gini Index

$$\text{Gini}(D) = 1 - \sum_{i=1}^C p_i^2$$

- p_i = proportion of class i in dataset D
- Lower Gini = purer node

B. For Regression: Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y})^2$$

- y_i = true value,
- $\hat{y}$ = predicted mean value

7. Advantages of CART

- Supports **both classification and regression**
- Handles **numerical and categorical data**
- Creates **binary splits**, making the tree more consistent
- Includes **pruning** (post-pruning with cost-complexity pruning)
- Often more **robust** than ID3/C4.5

8. Disadvantages

- Can still **overfit** without pruning
- Binary splits may lead to **deeper trees**
- Sensitive to **small data changes** (can change structure)
- May be **unstable** (small data changes can lead to different trees)
- Biased toward features with **more split points**
- Trees may become **complex and hard to interpret** if not constrained

Example

CGPA	Interactive	Practical Knowledge	Comm Skills	Job Offer
≥ 9	Yes	Very good	Good	Yes
≥ 8	No	Good	Moderate	Yes
≥ 9	No	Average	Poor	No
< 8	No	Average	Good	No
≥ 8	Yes	Good	Moderate	Yes
≥ 9	Yes	Good	Moderate	Yes
< 8	Yes	Good	Poor	No
≥ 9	No	Very good	Good	Yes
≥ 8	Yes	Good	Good	Yes
≥ 8	Yes	Average	Good	Yes

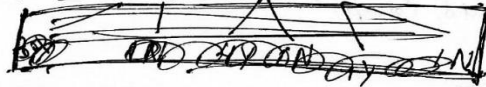
Answer

$$\text{Gini Index} = 1 - \sum_{i=1}^m p_i^2$$

$$\text{Gini(Table)} = 1 - \left(\frac{7}{10}\right)^2 - \left(\frac{3}{10}\right)^2 = 1 - 0.49 - 0.09 \\ = 0.42$$

① Attribute CGPA

Values of CGPA = $\{ \geq 9, \geq 8, < 8 \}$



CGPA	Yes	No
≥ 9	3	1
≥ 8	4	0
< 8	0	2

we need to find Gini-index
of each subset of CGPA
3 values of CGPA
 $\therefore 2^3$ subsets.

Subsets = $\{ \{ \}, \{ \geq 9 \}, \{ \geq 8 \}, \{ < 8 \}, \{ \geq 9, \geq 8 \}, \{ \geq 9, < 8 \}, \{ \geq 8, < 8 \}, \{ \geq 9, \geq 8, < 8 \} \}$

$$G I(T, \{ \geq 9, \geq 8 \}) = 1 - \left(\frac{3+4}{8} \right)^2 - \left(\frac{1}{8} \right)^2 = 1 - \frac{49}{64} - \frac{1}{64}$$

$$= \frac{64 - 49 - 1}{64} = \frac{14}{64} = 0.21875$$

$$G I(T, \{ \leq 8 \}) = 1 - \left(\frac{0}{2} \right)^2 - \left(\frac{2}{2} \right)^2 = 0$$

$$G I(T, \{ \geq 9, \geq 8 \}, \leq 8 \}) = \frac{8}{10} \times 0.21875 + \frac{2}{10} \times 0$$

$$= 0.175$$

$$G I(T, \{ \geq 9, < 8 \}) = 1 - \left(\frac{3}{6} \right)^2 - \left(\frac{3}{6} \right)^2 = 0.5$$

$$G I(T, \{ \geq 8 \}) = 1 - \left(\frac{4}{4} \right)^2 - \left(\frac{0}{4} \right)^2 = 0$$

$$G I(T, \{ \geq 9, < 8 \}, \geq 8 \}) = \frac{6}{10} \times 0.5 + \frac{4}{10} \times 0 = 0.3$$

$$G I(T, \{ \geq 8, < 8 \}) = 1 - \left(\frac{4}{6} \right)^2 - \left(\frac{2}{6} \right)^2 = 0.4444$$

$$G I(T, \{ \geq 9 \}) = 1 - \left(\frac{3}{4} \right)^2 - \left(\frac{1}{4} \right)^2 = 0.375$$

$$G I(T, \{ \geq 9, < 8 \}, \geq 9 \}) = \frac{6}{10} \times 0.4444 + \frac{4}{10} \times 0.375$$

$$= 0.41664$$

Now, Subsets	G.I	All close min.
$\{ \geq 9, \geq 8 \}, \leq 8 \}$	0.175	G.I for best splitting.
$\{ \geq 9, < 8 \}, \geq 8 \}$	0.3	
$\{ < 8, \geq 8 \}, \geq 9 \}$	0.41664	

$\therefore CGPA = \{ \geq 9, \geq 8 \}, \leq 8 \}$ is selected.

$$\text{New Abini (CGPA)} = Gini(T) - Gini(T, CGPA)$$

$$= 0.42 - 0.175 = \boxed{0.2445}$$

② Attribute: Int.

Int	Yes	No
Yes	5	1
No	2	2

Subsets {Yes, No}

$$GI(T, \text{Yes}) = 1 - \left(\frac{5}{6}\right)^2 - \left(\frac{1}{6}\right)^2 = 0.278$$

$$GI(T, \text{No}) = 1 - \left(\frac{1}{2}\right)^2 - \left(\frac{1}{2}\right)^2 = 0.5$$

$$GI(T, \{N\}) = \frac{6}{10} \times 0.278 + \frac{4}{10} \times 0.5 = 0.3668$$

$$\Delta Gini(\text{Int}) = 0.42 - 0.3668 = 0.0532$$

③ $\Delta Gini(\text{pro c}) = 0.1146$

④ $\Delta Gini(\text{comm}) = 0.2445$

⑤ $\Delta Gini(\text{CGPA}) = 0.2445$

we can choose
between (any one)

Let's choose CGPA.



CGPA	Int	pro c.	comm.	job.
≥ 9	Y	very good	good	Y
≥ 8	N	good	medium	Y
≥ 9	N	bad	poor	N
≥ 8	Y	good	mod.	Y
≥ 9	Y	good	mod	Y
≥ 8	N	very bad	good	Y
≥ 8	Y	good	good	Y
≥ 8	Y	avg	good	Y

$$\text{Gini}(T) = 1 - \left(\frac{7}{8}\right)^2 - \left(\frac{1}{8}\right)^2 = 0.21875$$

① Int

Int	Yes	No
Y	5	0
N	2	1

$$\text{Gini}(T, \text{Yes}) = \frac{1}{8} - \left(\frac{5}{8}\right)^2 - \left(\frac{0}{8}\right)^2 = 0$$

$$\text{Gini}(T, \text{No}) = 1 - \left(\frac{2}{3}\right)^2 - \left(\frac{1}{3}\right)^2 = 0.444$$

$$\begin{aligned} \text{Gini}(T, (Y, N)) &= \left(\frac{5}{8}\right)^2 \times 0 + \left(\frac{3}{8}\right)^2 \times 0.444 \\ &= 0.06244 \end{aligned}$$

$$\Delta \text{Gini}(T) = 0.21875 - 0.06244 = 0.15631$$

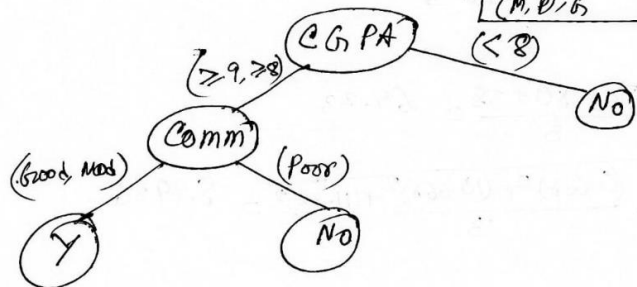
② Proc

$$\Delta \text{Gini}(P) = 0.0934$$

③ Comm

$$\Delta \text{Gini}(C) = 0.2184$$

(V6, G) A	0.125
(V6, A) G	0.1875
(A, G) V6	0.2085
(G, M) P	0
(G, P) M	0.2
(M, P) G	0.1875



Aspect	Gini Index	Entropy
Definition	Measures the impurity of a dataset as the probability of misclassification.	Measures the impurity based on information theory — amount of information needed to classify.
Used In	Used by CART algorithm.	Used by ID3 and C4.5 algorithms.
Formula	$\text{Gini}(D) = 1 - \sum_{i=1}^C p_i^2$	$\text{Entropy}(S) = - \sum_{i=1}^k p_i \log_2 p_i$
Range	0 (pure) to < 0.5 (for 2 classes)	0 (pure) to $\log_2(C)$, C = number of classes
Computation	Faster and simpler to calculate.	Slightly more computationally expensive.
Split Bias	Prefers splits with larger class dominance .	Prefers balanced splits (more sensitive to class distribution).
Units	No specific units.	Measured in bits (information gain).

Aspect	CART	ID3
Definition	Decision Tree algorithm that uses Gini Index and supports classification and regression.	Decision Tree algorithm that uses Entropy and Information Gain for classification only.
Tree Type	Always creates binary splits (2 branches).	Creates multi-way splits (branches = number of attribute values).
Splitting Criterion	Uses Gini Index .	Uses Information Gain (Entropy) .
Supports Regression	Yes .	No , only classification.
Numerical Attributes	Directly supported.	Not directly supported (need discretization).
Pruning	Includes post-pruning (cost-complexity pruning).	No pruning (builds full tree).
Output Tree Depth	Can be deeper due to binary splits.	Usually shallower with multi-way splits.
Overfitting	Handles it better with pruning.	Prone to overfitting .

Regression Tree (type of decision tree)

A **Regression Tree** is a type of **Decision Tree** used for **predicting continuous (real-valued) outputs** instead of categorical labels.

- Unlike classification trees (which predict classes like *Yes/No*), regression trees predict values like *salary, temperature, profit*, etc.
- Goal:** To partition the input space into regions where the **target variable's variance is minimized** within each region, and the **mean value** of the region is used for prediction.

Example

S.No.	Assessment	Assignment	Project	Result (%)
1	Good	Yes	Yes	95
2	Average	Yes	No	70
3	Good	No	Yes	75
4	Poor	No	No	45
5	Good	Yes	Yes	98
6	Average	No	Yes	80
7	Good	No	No	75
8	Poor	Yes	Yes	65
9	Average	No	No	58
10	Good	Yes	Yes	89

Answer

$$\text{Mean} = \frac{95 + 70 + 75 + 45 + 98 + 80 + 75 + 65 + 58 + 89}{10} = 75$$

$$Sd = \sqrt{\frac{\sum (x_i - \bar{x})^2}{N}} = \sqrt{\frac{(20)^2 + 5^2 + 0^2 + 30^2 + 23^2 + 5^2 + 0^2 + 10^2 + 12^2 + 14^2}{10}} = 15.6971$$

i) Assessment values / Good, Avg, Poor

① Good

$$\text{Mean} = \frac{95 + 75 + 98 + 75 + 89}{5} = 86.4$$

$$Sd = \sqrt{\frac{(8.6)^2 + (1.4)^2 + (11.6)^2 + (1.4)^2 + (2.6)^2}{5}} = 9.2488$$

② Avg

$$\text{Mean} = \frac{70 + 80 + 58}{3} = 69.33$$

$$Sd = \sqrt{\frac{(0.66)^2 + (10.66)^2 + (11.33)^2}{3}} = 8.9926$$

③ Poor

$$\text{Mean} = \frac{45 + 65}{2} = 55$$

$$Sd = \sqrt{\frac{10^2 + 10^2}{2}} = 10$$

$$\Delta Sd = 15.6971 - \left(\frac{5}{10} \times 9.2488 + \frac{3}{10} \times 8.9926 + \frac{2}{10} \times 10 \right)$$

$$= 15.6971 - 9.57218 = \boxed{6.12492}$$

ii) Assignment

values (Yes, No)

① Yes

$$\text{Mean} = \frac{95 + 70 + 98 + 65 + 89}{5} = 83.4$$

$$Sd = \sqrt{\frac{(11.6)^2 + (3.4)^2 + (14.6)^2 + (8.4)^2 + (4.6)^2}{5}} = 13.3192$$

② No

$$\text{Mean} = \frac{75 + 45 + 80 + 65 + 58}{5} = 66.6$$

$$Sd = \sqrt{\frac{(8.4)^2 + (2.6)^2 + (13.4)^2 + (8.4)^2 + (8.6)^2}{5}} = 13.1240$$

$$\Delta Sd = 15.6971 - \left(\frac{2}{5} \times 13.3192 + \frac{3}{5} \times 13.1240 \right)$$

$$= \boxed{2.4755}$$

③ Prb.

$$\text{① Sd of Yes} = 12.6 \quad \text{② Sd of No} = 13.39 \quad \Delta Sd = \boxed{3.63}$$

We chose largest S.d. which is 6.1249.



Naïve Bayes

Bayes Classifier

The Bayes classifier is a **theoretical optimal classifier** that minimizes the **probability of classification error**. It uses **posterior probabilities** calculated via Bayes' theorem and assigns a sample to the class with the **highest posterior**. **Characteristics:**

- Requires knowledge of class priors $P(\omega_i)$ and likelihoods $P(x|\omega_i)$
- Minimizes classification error (optimal in theory)
- Not always practical due to the need for **complete probability distributions**

Decision Rule:

Given an input vector x , the Bayes classifier assigns it to the class ω_i such that:

$$\omega_{\text{assigned}} = \arg \max_{\omega_i} P(\omega_i | x)$$

[arg max finds the argument(ω_i) for which the probability is maximum]

Using Bayes' theorem:

$$P(\omega_i | x) = \frac{P(x | \omega_i) \cdot P(\omega_i)}{P(x)}$$

Since $P(x)$ is the same for all classes, the decision depends on:

$$\omega_{\text{assigned}} = \arg \max_{\omega_i} P(x | \omega_i) \cdot P(\omega_i)$$

Naïve Bayes Classifier

The Naïve Bayes classifier is a **simplified version of the Bayes classifier**. It assumes that all features are **conditionally independent** given the class label.

Formula:

If $x = (x_1, x_2, \dots, x_n)$ is the feature vector, then:

$$P(x | \omega_i) = \prod_{j=1}^n P(x_j | \omega_i)$$

So, the decision rule becomes:

$$\omega_{\text{assigned}} = \arg \max_{\omega_i} P(\omega_i) \cdot \prod_{j=1}^n P(x_j | \omega_i)$$

Characteristics:

- Easy to implement
- Performs well in many real-world applications despite the independence assumption
- Requires fewer parameters than the full Bayes classifier
- Often used in text classification, spam detection, sentiment analysis